PaySafe — Payment System

Ledger + Double Entry + Idempotency

Student Name: Naaz Fatema Ahmedi

Roll Number: 150096724038

Subject: System Design

Academic Year: 2024-2028

Problem No.: 05

Problem Statement

Payment systems must process transactions reliably and prevent problems such as duplicate payments, incorrect balances and incomplete transactions.

The objective of this project is to design and implement a simplified payment system that uses:

- Double-entry ledger
- Idempotency
- Transaction status tracking
- Failure handling
- Audit logging

2. Objectives

- Process payment requests through REST APIs.
- Prevent duplicate payments using idempotency keys.
- Maintain financial records using double-entry accounting.
- Track payment status from creation to completion.
- Handle failed payment requests.
- Maintain audit records.
- Provide a scalable architecture that can be extended for larger systems.

3. Proposed Solution

PaySafe is a Python-based payment processing system developed using FastAPI and SQLite.

A payment request follows this flow:

Client → FastAPI → Idempotency Check → Payment Service → Payment Gateway → Ledger → Database → Response

For every successful payment, the system creates:

1 DEBIT + 1 CREDIT

The same idempotency key cannot create another payment for the same request.

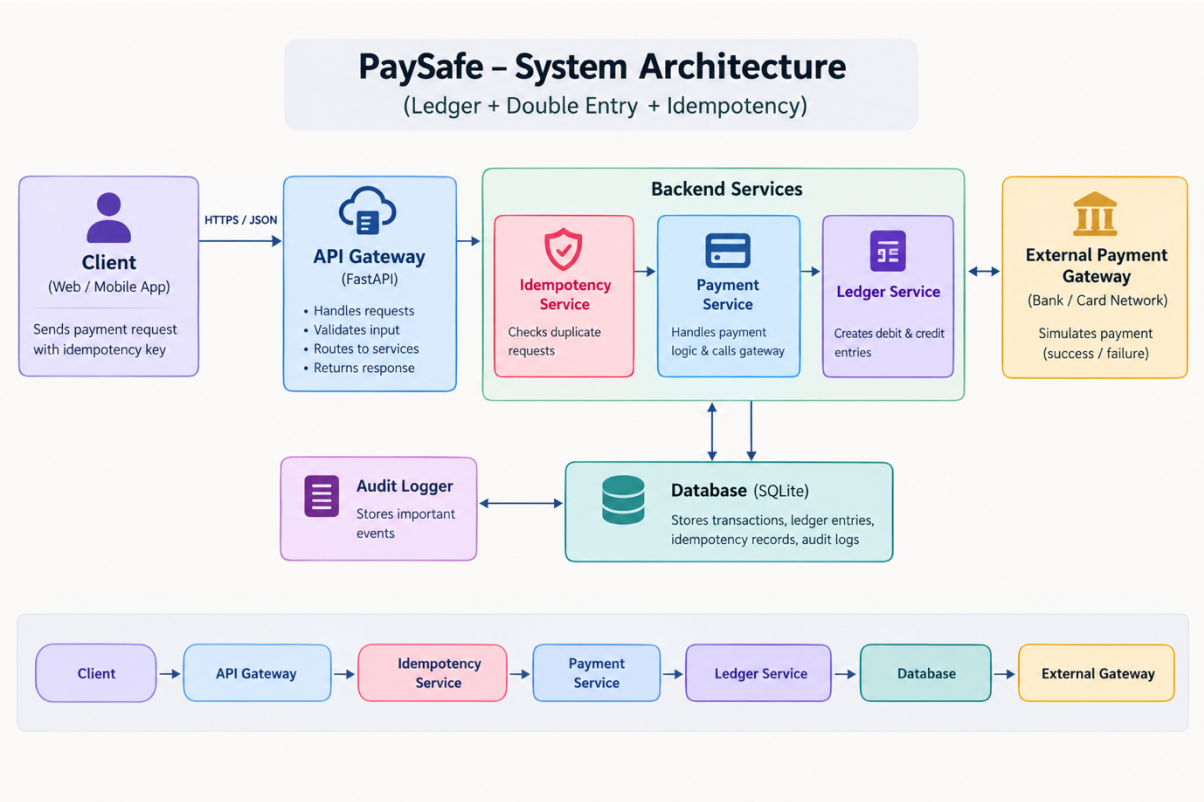
The external payment gateway is simulated for this academic implementation.

4. System Architecture

Architecture Components

Component	Purpose
Client	Sends payment requests
FastAPI	Provides REST APIs
Idempotency Service	Prevents duplicate requests
Payment Service	Handles payment workflow
Gateway Service	Simulates external payment gateway
Ledger Service	Records debit and credit entries
Audit Logger	Records important events
SQLite Database	Stores system data

Architecture Diagram



5. System Workflow

Payment Flow



Duplicate Request

If the same request is sent again with the same idempotency key, PaySafe returns the previously stored result instead of creating another transaction.

6. Modules

Module	Responsibility
Payment Service	Main payment processing logic
Ledger Service	Double-entry accounting
Idempotency Service	Duplicate request prevention
Gateway Service	Simulated payment gateway
Audit Service	Event logging
Database	Stores transactions and account data
API	Handles client requests

7. Database Design

PaySafe uses SQLite for the student-level implementation.

Main Tables

Table	Purpose
Accounts	Stores customer and merchant accounts
Transactions	Stores payment details and status
Ledger Entries	Stores debit and credit records
Idempotency Records	Stores request keys and previous responses
Audit Logs	Stores important system events

Double-Entry Example

For a payment of ₹500:

Account	Entry	Amount
Customer	DEBIT	₹500
Merchant	CREDIT	₹500

Debit = Credit

This helps maintain consistency in the financial records.

8. Technology Stack

Technology	Use
Python	Backend development
FastAPI	REST API
Pydantic	Data validation
SQLite	Database
Uvicorn	Server
Pytest	Testing
Matplotlib	Architecture diagram

9. API Endpoints

Method	Endpoint	Purpose
POST	/payments	Create payment
GET	/payments/{id}	Get payment details
POST	/refunds	Process refund
GET	/accounts/{id}/balance	Check balance
GET	/ledger/{id}	View ledger
GET	/health	Check system health

10. Key Implementation Features

Idempotency

- Each payment request requires an idempotency key.
- If the same key and request are received again, the original response is returned.
- This prevents accidental duplicate payments.

Double-Entry Ledger

- Every successful payment creates both a debit and a credit entry of the same amount.

Transaction Status

- The system tracks payment states such as:
- **PENDING → PROCESSING → SUCCESS / FAILED**
- Refunded transactions are also supported.

Failure Handling

- If the simulated payment gateway fails, the transaction is marked as **FAILED** and no successful ledger entry is created.

11. Testing

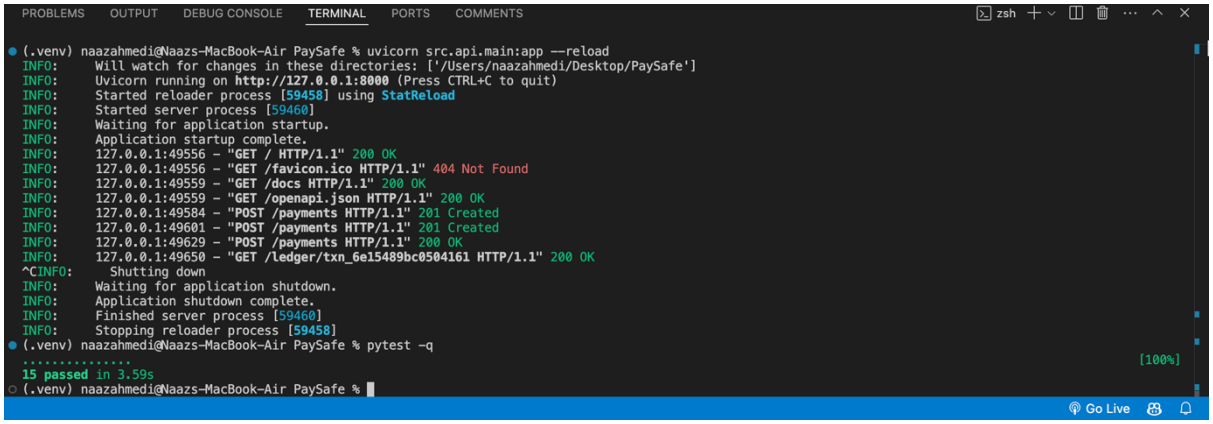
The project includes automated tests using Pytest.

The tests verify:

- Payment creation
- Payment retrieval
- Duplicate request handling
- Idempotency validation
- Double-entry ledger
- Account balances
- Transaction status

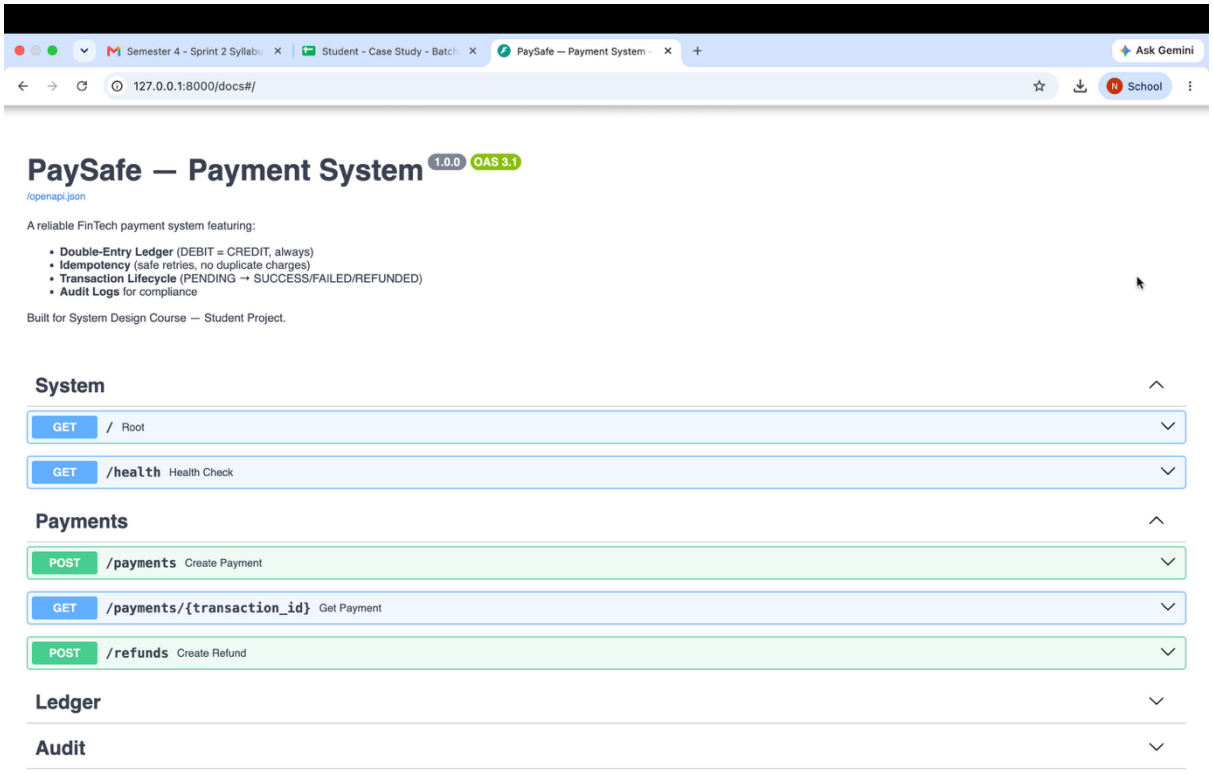
Test Result

15 tests passed successfully.

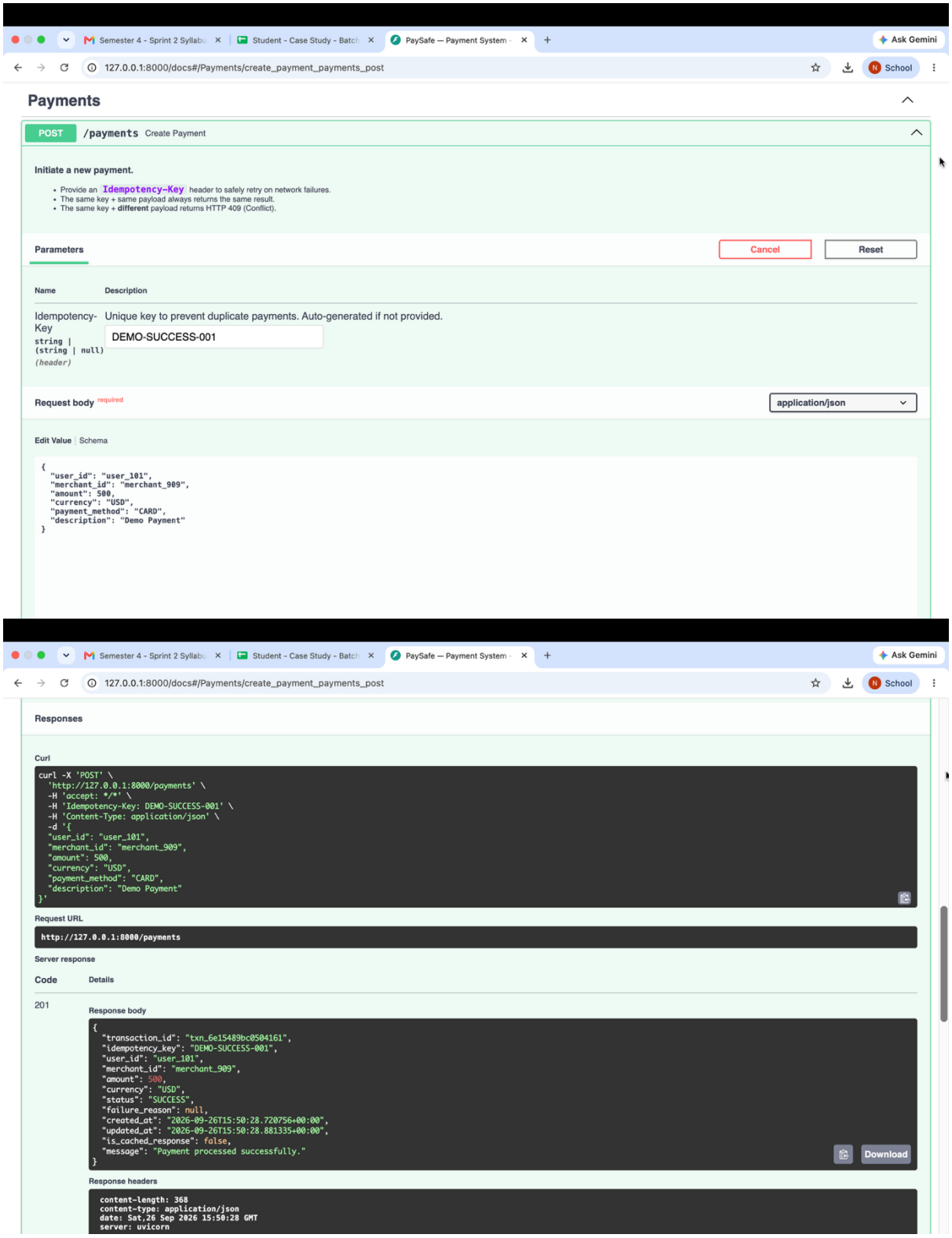


12. Screenshots

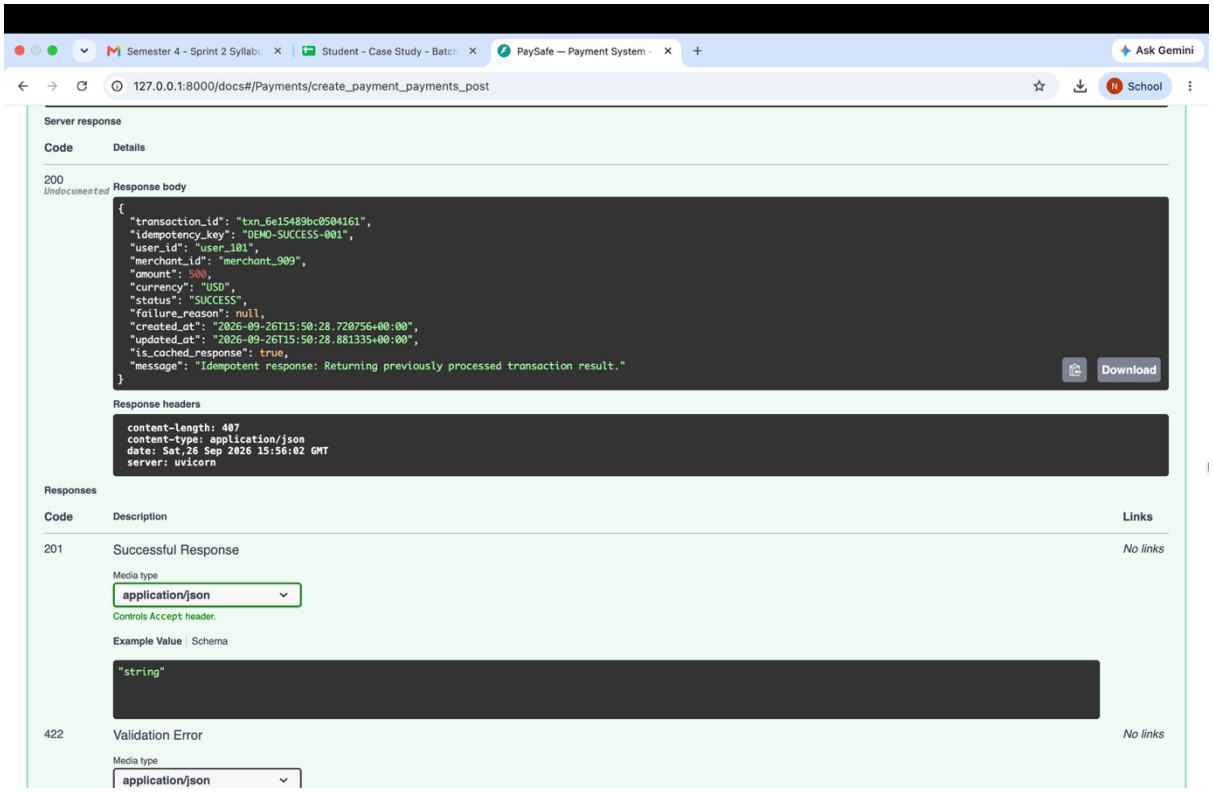
Screenshot 1 — API Documentation



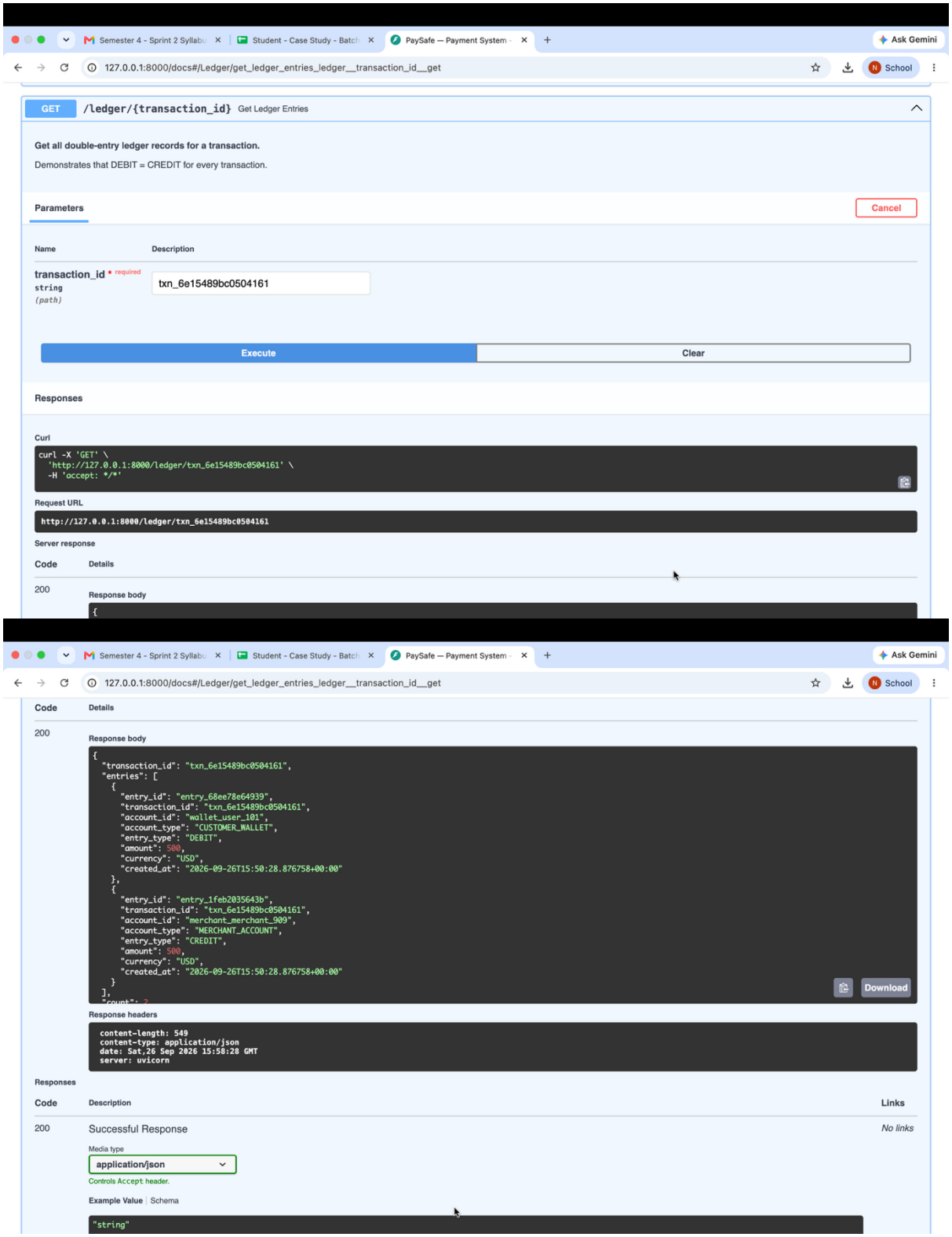
Screenshot 2 — Successful Payment



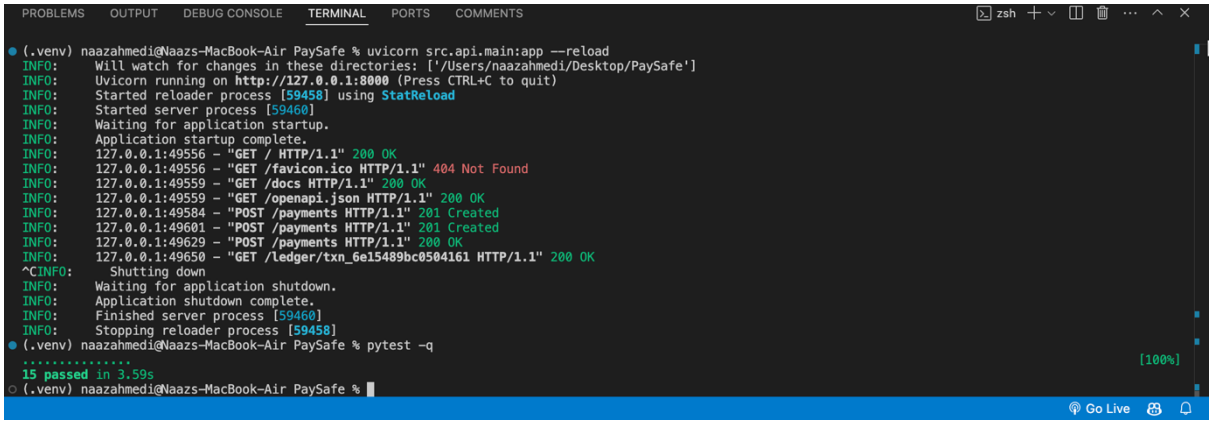
Screenshot 3 — Idempotency



Screenshot 4 — Ledger



Screenshot 5 — Test Results



13. Advantages and Limitations

Advantages

- Prevents duplicate payments.
- Maintains debit and credit records.
- Tracks transaction status.
- Handles payment failures.
- Provides REST APIs.
- Includes automated testing.

Limitations

- SQLite is used for the academic implementation.
- External payment gateway is simulated.
- Authentication is not implemented.
- Distributed deployment is not implemented.

14. Future Scope

The system can be extended using:

- PostgreSQL for production-scale database requirements.
- Redis for caching and distributed idempotency.
- Load balancers for horizontal scaling.
- Authentication and authorization.
- Real payment gateway integration.
- Monitoring and centralized logging.
- Fraud detection mechanisms

15. Conclusion

PaySafe demonstrates the basic design of a reliable payment processing system using **idempotency, double-entry accounting, transaction management and failure handling**.

The project successfully implements the core payment workflow while providing a foundation that can be extended into a larger distributed payment architecture.